

Student Placement Prediction

In this we have the students records of their academic performance and their placement details and now we need to build the predictive model to analyze the student placement.

Problem Statement: Students career details and their performance based on that we should predict the future outcomes.

Problem Objective: Finding the most likely way to get placed.

Problem Constraints: Finding way to get high paying job.

INTRODUCTION

We got the students data with their academic, technical, and career-related attributes and each of the student has unique records and has shown their interest in different areas, Therefore now we need to help the college by providing them an proper data-driven career guidance and training design and finally how to improve Placement with high paying job.

Let's see to the data and find some insights,

Importing Libraries

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

Here we are importing the basic required libraries for our project.

Data Collection and analysis

```
In [ ]: PS = pd.read_csv(r"/content/student_placement_prediction_dataset (1).csv")
```

Uploading the students data into variable called 'PS'.

```
In [ ]: PS.info()
PS.shape
PS.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 26 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   student_id                            10000 non-null  int64
1   age                                    10000 non-null  int64
2   gender                                10000 non-null  object
3   cgpa                                   10000 non-null  float64
4   branch                                10000 non-null  object
5   college_tier                           10000 non-null  object
6   internships_count                      10000 non-null  int64
7   projects_count                        10000 non-null  int64
8   certifications_count                  10000 non-null  int64
9   coding_skill_score                    10000 non-null  float64
10  aptitude_score                        10000 non-null  float64
11  communication_skill_score             10000 non-null  float64
12  logical_reasoning_score                10000 non-null  float64
13  hackathons_participated                10000 non-null  int64
14  github_repos                          10000 non-null  int64
15  linkedin_connections                  10000 non-null  int64
16  mock_interview_score                  10000 non-null  float64
17  attendance_percentage                  10000 non-null  float64
18  backlogs                              10000 non-null  int64
19  extracurricular_score                 10000 non-null  float64
20  leadership_score                      10000 non-null  float64
21  volunteer_experience                   10000 non-null  object
22  sleep_hours                           10000 non-null  float64
23  study_hours_per_day                   10000 non-null  float64
24  placement_status                      10000 non-null  object
25  salary_package_lpa                    10000 non-null  float64
dtypes: float64(12), int64(9), object(5)
memory usage: 2.0+ MB
```

	student_id	age	gender	cgpa	branch	college_tier	internships_c
0	1	24	Male	7.53	IT	Tier 2	4
1	2	21	Male	7.92	CSE	Tier 2	1
2	3	22	Female	8.60	EEE	Tier 1	0
3	4	24	Male	6.68	CSE	Tier 1	0
4	5	20	Female	8.43	IT	Tier 3	1

5 rows × 26 columns

Our Student data set contains of 26 columns and 10000 entries

and this data set contains 24 input columns and 2 output columns.

This data set contains two types of output columns,

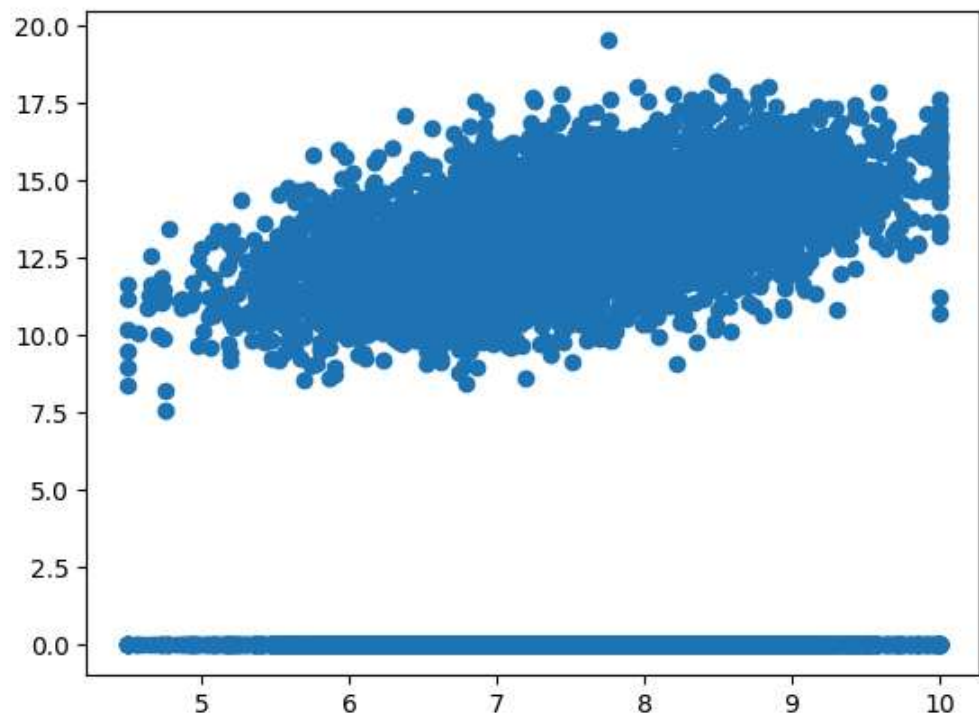
Classification - "placement_status" (It contains students placed or not only two unique values, therefor it is an classification)

Regression - "salary_package_lpa" (It contains the students salary as float, that is different from each and every student, therefor its an regression)

EDA (Exploratory Data Analysis)

```
In [ ]: plt.scatter(x = PS['cgpa'], y = PS['salary_package_lpa'])
```

<matplotlib.collections.PathCollection at 0x7e947e141df0>



We have been plotted an scatter plot for the "cgpa" and the "salary_package_lpa" because the cgpa is the most important one for the placement and based upon cgpa most of the salaries are been fixed.

```
In [ ]: duplicate = PS.duplicated()  
duplicate
```

	0
0	False
1	False
2	False
3	False
4	False
...	...
9995	False
9996	False
9997	False
9998	False
9999	False

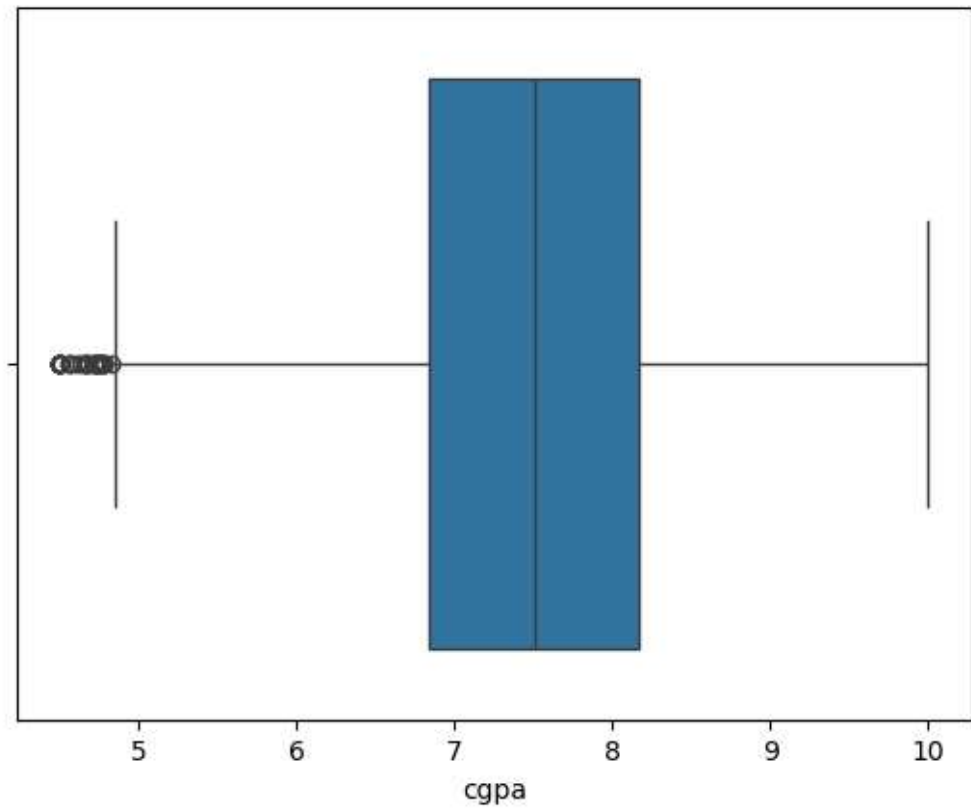
10000 rows × 1 columns

dtype: bool

Here we are checking that any duplicates in our data set and we got all negative.

```
In [ ]: sns.boxplot(x = PS['cgpa'])
```

<Axes: xlabel='cgpa'>

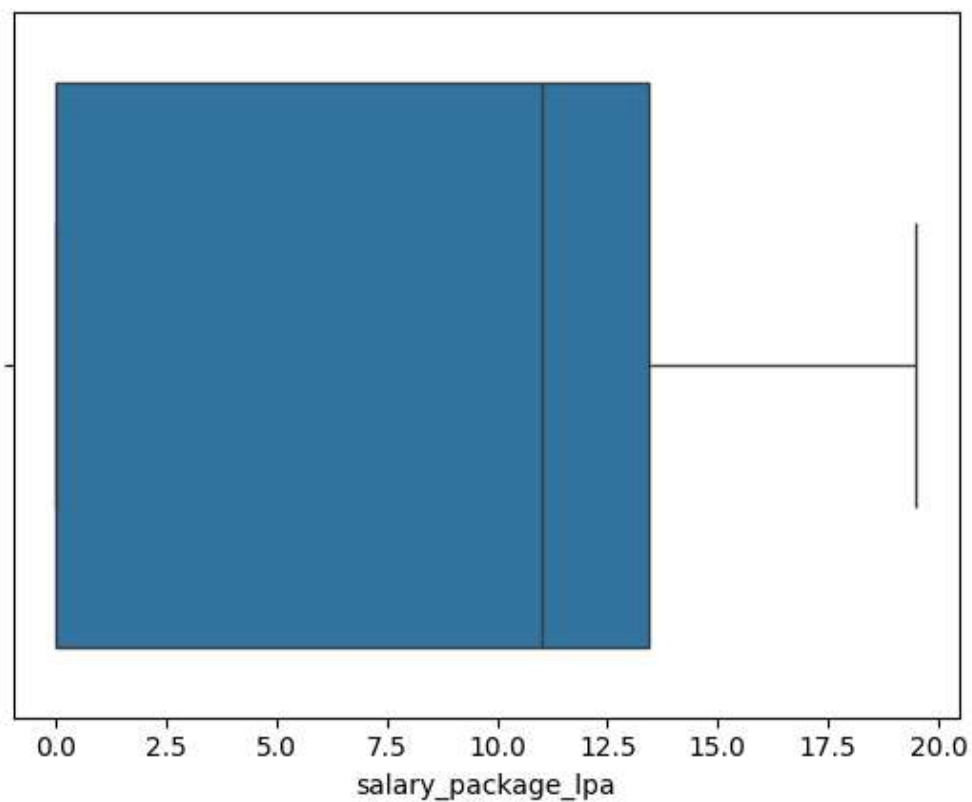


To check any outliers in the "cgpa" columns we use boxplot.

Here in this some points are below 5 and it looks like outliers but it is that some students has less than 5 cgpa.

```
In [ ]: sns.boxplot(x = PS['salary_package_lpa'])
```

<Axes: xlabel='salary_package_lpa'>



As well as "cgpa" now we had an boxplot for "salary_package_lpa"

and here there is no outliers.

In []:

PS.isnull().sum()

	0
student_id	0
age	0
gender	0
cgpa	0
branch	0
college_tier	0
internships_count	0
projects_count	0
certifications_count	0
coding_skill_score	0
aptitude_score	0
communication_skill_score	0
logical_reasoning_score	0
hackathons_participated	0
github_repos	0
linkedin_connections	0
mock_interview_score	0
attendance_percentage	0
backlogs	0
extracurricular_score	0
leadership_score	0
volunteer_experience	0
sleep_hours	0
study_hours_per_day	0
placement_status	0
salary_package_lpa	0

dtype: int64

Finding null values using "isnull"

and there is no null values in our data.

Classification

Data Preprocessing for classification

```
In [ ]: from sklearn.pipeline import Pipeline
        from sklearn.preprocessing import LabelEncoder

        import statsmodels.api as sm
        from sklearn.model_selection import train_test_split
```

```
In [ ]: PS.at[5033, 'salary_package_lpa'] = 0.00
```

We had find that there are unplaced students and for them there will be empty "salary_package_lpa" column.

For that we had mentiond 0.00 for those students.

```
In [ ]: PS['salary_package_lpa'].head(10)
PS['salary_package_lpa'].tail(10)
```

	salary_package_lpa
9990	0.00
9991	0.00
9992	0.00
9993	11.17
9994	0.00
9995	0.00
9996	13.18
9997	0.00
9998	0.00
9999	14.55

dtype: float64

Therefor all the students has some values in "salary_package_lpa" column.

```
In [ ]: X1 = PS.drop(['student_id', 'placement_status', 'salary_package_lpa'],
axis=1)
X1
```

	age	gender	cgpa	branch	college_tier	internships_count	p
0	24	Male	7.53	IT	Tier 2	4	6
1	21	Male	7.92	CSE	Tier 2	1	3
2	22	Female	8.60	EEE	Tier 1	0	1
3	24	Male	6.68	CSE	Tier 1	0	2
4	20	Female	8.43	IT	Tier 3	1	4
...	...	...	...	...	...	...	...
9995	20	Male	8.29	CSE	Tier 2	4	5
9996	22	Male	7.59	CSE	Tier 2	2	5
9997	19	Male	6.72	CSE	Tier 2	1	4
9998	20	Male	8.53	ECE	Tier 2	0	3
9999	19	Female	7.88	CSE	Tier 1	0	2

10000 rows × 23 columns



Creating an new variable for the input datas alone as (X1) by dropping the output columns.

```
In [ ]: X1= pd.get_dummies(X1, columns=['gender', 'branch', 'college_tier',
'volunteer_experience'], drop_first=True)
```

Here we are been converted these columns as intigers using "get_dummies" and created some new columns.

```
In [ ]: X1 = X1.astype(float)
```

Convert it into float datatype.

```
In [ ]: print(X1.shape)
print(X1.isnull().sum().sum())
```

```
(10000, 28)
0
```

By including some new columns we have 28 columns in X1

With zero null values.

```
In [ ]: Y1 = PS[['placement_status']]  
Y1
```

	placement_status
0	Not Placed
1	Not Placed
2	Placed
3	Not Placed
4	Placed
...	...
9995	Not Placed
9996	Placed
9997	Not Placed
9998	Not Placed
9999	Placed

10000 rows × 1 columns

creating an variable for first output column.

```
In [ ]: numeric_features = X1.select_dtypes(exclude = ['object']).columns
numeric_features
```

```
Index(['age', 'cgpa', 'internships_count', 'projects_count',
      'certifications_count', 'coding_skill_score',
      'aptitude_score',
      'communication_skill_score', 'logical_reasoning_score',
      'hackathons_participated', 'github_repos',
      'linkedin_connections',
      'mock_interview_score', 'attendance_percentage', 'backlogs',
      'extracurricular_score', 'leadership_score', 'sleep_hours',
      'study_hours_per_day', 'gender_Male', 'branch_Civil',
      'branch_ECE',
      'branch_EEE', 'branch_IT', 'branch_Mechanical',
      'college_tier_Tier 2',
      'college_tier_Tier 3', 'volunteer_experience_Yes'],
      dtype='object')
```

This will exclude all the columns which has "object" as their datatype.

```
In [ ]: numeric_features1 = X1.select_dtypes(include = ['int64']).columns
numeric_features1

numeric_features2 = X1.select_dtypes(include = ['float64']).columns
numeric_features2
```

```
Index(['age', 'cgpa', 'internships_count', 'projects_count',
      'certifications_count', 'coding_skill_score',
      'aptitude_score',
      'communication_skill_score', 'logical_reasoning_score',
      'hackathons_participated', 'github_repos',
      'linkedin_connections',
      'mock_interview_score', 'attendance_percentage', 'backlogs',
      'extracurricular_score', 'leadership_score', 'sleep_hours',
      'study_hours_per_day', 'gender_Male', 'branch_Civil',
      'branch_ECE',
      'branch_EEE', 'branch_IT', 'branch_Mechanical',
      'college_tier_Tier 2',
      'college_tier_Tier 3', 'volunteer_experience_Yes'],
      dtype='object')
```

It divides the columns as two different which one has int as datatype and another has float as sayatype.

```
In [ ]: le = LabelEncoder()
```

Lets do some encoding for the "placement_status" column.

```
In [ ]: Y = le.fit_transform(PS['placement_status'])
        Y

        array([0, 0, 1, ..., 0, 0, 1])
```

Lets fit and transform the label encoder.

```
In [ ]: X1_train, X1_test, Y_train, Y_test = train_test_split(X1, Y,
                                                             test_size=0.2,
                                                             random_state=42)

        print(X1_train.shape)
        print(X1_test.shape)

        (8000, 28)
        (2000, 28)
```

Splitting up the data into test and train model for the machine.

It will learn the model using train data and use test data for evaluation.

Logistic Regression

```
In [ ]: from sklearn.linear_model import LogisticRegression

        from sklearn import metrics
        from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
        accuracy_score
        from sklearn.metrics import roc_curve
        from sklearn.metrics import classification_report
```

Importing the required libraries for the Logistic regression model.

This model is for classification there by we take all inputs as X1 and only "placement_status" as Y1.

```
In [ ]: logisticmodel = sm.Logit(Y_train, X1_train).fit()

        Optimization terminated successfully.
        Current function value: 0.674773
        Iterations 4
```

Fiting the datas into the logisticmodel.

```
In [ ]: Y_pred_train = logisticmodel.predict(X1_train)
        Y_pred_train
```

	0
9254	0.573635
1561	0.476393
1670	0.666359
6087	0.559799
6669	0.538593
...	...
5734	0.353690
5191	0.492820
5390	0.585743
860	0.421809
7270	0.529720

8000 rows × 1 columns

dtype: float64

Creating the new column "Y_pred_train" to predict the insights from train data set.

```
In [ ]: Y_train = pd.DataFrame(Y_train, columns=['placement_status'])

        Y_train["pred"] = np.zeros(len(Y_train))
```

Y_train is full of arrays and we need to convert it into dataframes thereby we can add some more columns to it

and new column as "pred" it will create column with full of zeros.

```
In [ ]: optimal_threshold = 0.5
Y_train['pred'] = (Y_pred_train > optimal_threshold).astype(int)

print(Y_train.head(10))
```

	placement_status	pred
0	1	NaN
1	0	0.0
2	1	1.0
3	0	NaN
4	1	1.0
5	0	1.0
6	1	0.0
7	1	1.0
8	0	NaN
9	1	1.0

Here we have real predicted values in "pred" columns and we can see some "NaN".

```
In [ ]: print(Y_train['pred'].unique())
print(Y_train['pred'].isnull().sum())
```

```
[nan  0.  1.]
1613
```

We can see the unique values of "pred" column as some empty values around 1613 lets handle this using "fillna".

```
In [ ]: Y_train['pred'] = Y_train['pred'].fillna(0)
```

```
In [ ]: Y_train['pred'] = Y_train['pred'].astype(int)
```

After filling the empty columns now we have clear "pred" data set.

```
In [ ]: print(Y_train['pred'].unique())
print(Y_train['pred'].isnull().sum())
```

```
[0 1]
0
```

Result of Logistic Regression


```
In [ ]: print('Accuracy of Logistic Regression: ',
accuracy_score(Y_train['placement_status'], Y_train['pred']))

auc = metrics.roc_auc_score(Y_train['placement_status'], Y_pred_train)
print("AUC Score:", auc)

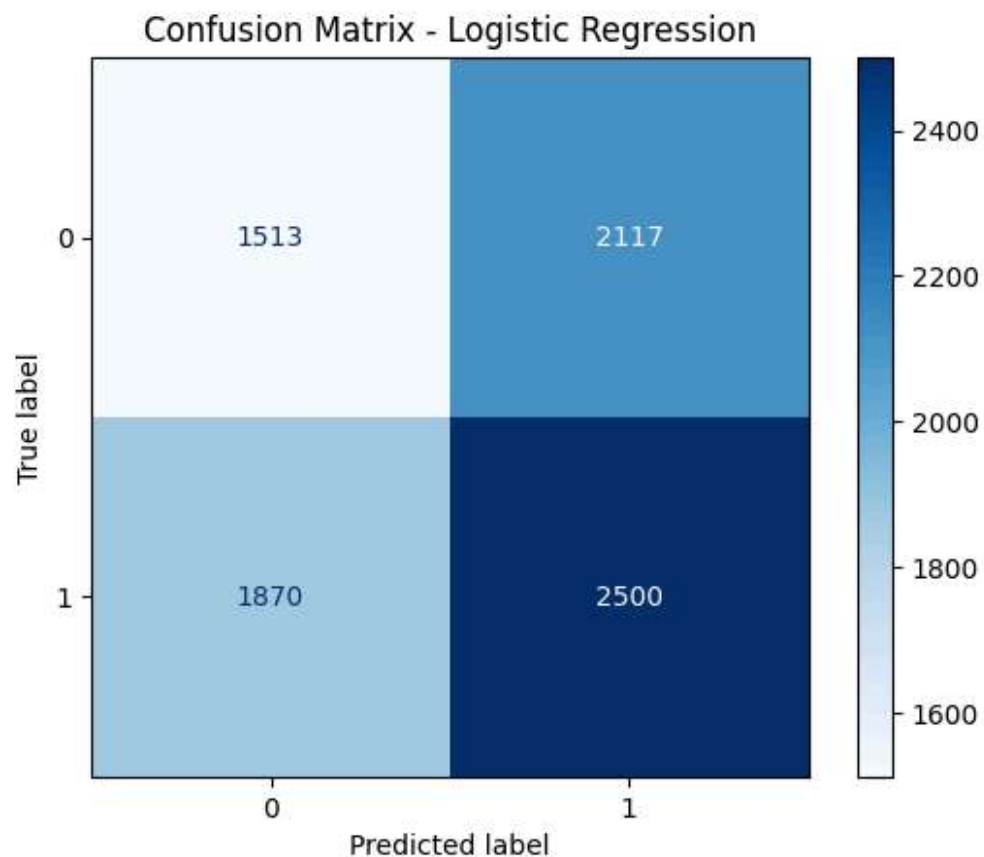
print('Classification report of Logistic Regression:\n'
,classification_report(Y_train['placement_status'], Y_train['pred']))

cm = confusion_matrix(Y_train['placement_status'], Y_train['pred'])
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```

```
Accuracy of Logistic Regression: 0.501625
AUC Score: 0.5941945143130913
Classification report of Logistic Regression:
              precision    recall  f1-score   support

      0         0.45         0.42         0.43         3630
      1         0.54         0.57         0.56         4370

 accuracy          0.50          0.50          0.50          8000
  macro avg         0.49         0.49         0.49          8000
 weighted avg         0.50         0.50         0.50          8000
```



Decision Tree

```
In [ ]: from sklearn.tree import DecisionTreeClassifier
```

Importing libraries for the decision tree model.

```
In [ ]: DT_model = DecisionTreeClassifier(random_state=42, max_depth=5)
DT_model.fit(X1_train, Y_train)
```

▼ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier(max_depth=5, random_state=42)

Initilizing the model with the DecisionTreeClassifier with the max_depth = 5.

```
In [ ]: Y_pred_train_DT_prob = np.array(DT_model.predict_proba(X1_train))[:, 1]

print(Y_pred_train_DT_prob[:5])

[[0.54207436 0.45792564]
 [0.43052838 0.56947162]]
```

```
In [ ]: Y_pred_test_DT_prob = np.array(DT_model.predict_proba(X1_test))[:, 1]

print(Y_pred_test_DT_prob[:5])

[[0.42885375 0.57114625]
 [0.414361   0.585639   ]]
```

Predicting the first 5 values to check the output is in proper way.

```
In [ ]: Y_pred_train_DT = DT_model.predict(X1_train)

Y_pred_test_DT = DT_model.predict(X1_test)
```

Predicting for the whole train and test .

```
In [ ]: from sklearn.model_selection import train_test_split

X1_train, X1_test, Y_train, Y_test = train_test_split(X1, Y,
                                                    test_size=0.2,
                                                    random_state=42,
                                                    stratify=Y)

print(X1_train.shape)
print(Y_train.shape)
print(Y_train[:5])

(8000, 28)
(8000,)
[0 0 0 1 1]
```

Splitting up the data again for our model.

Result of Decision tree

```

In [ ]: DT_model = DecisionTreeClassifier(random_state=42, max_depth=5)
DT_model.fit(X1_train, Y_train)

Y_pred_train_DT = DT_model.predict(X1_train)
Y_pred_test_DT  = DT_model.predict(X1_test)

print("Accuracy of Decision tree :", accuracy_score(Y_train,
Y_pred_train_DT))

print("Classification Report of Decision Tree :\n",
classification_report(Y_train, Y_pred_train_DT))

cm = confusion_matrix(Y_train, Y_pred_train_DT)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Greens')
plt.title("Confusion Matrix - Decision Tree")
plt.show()

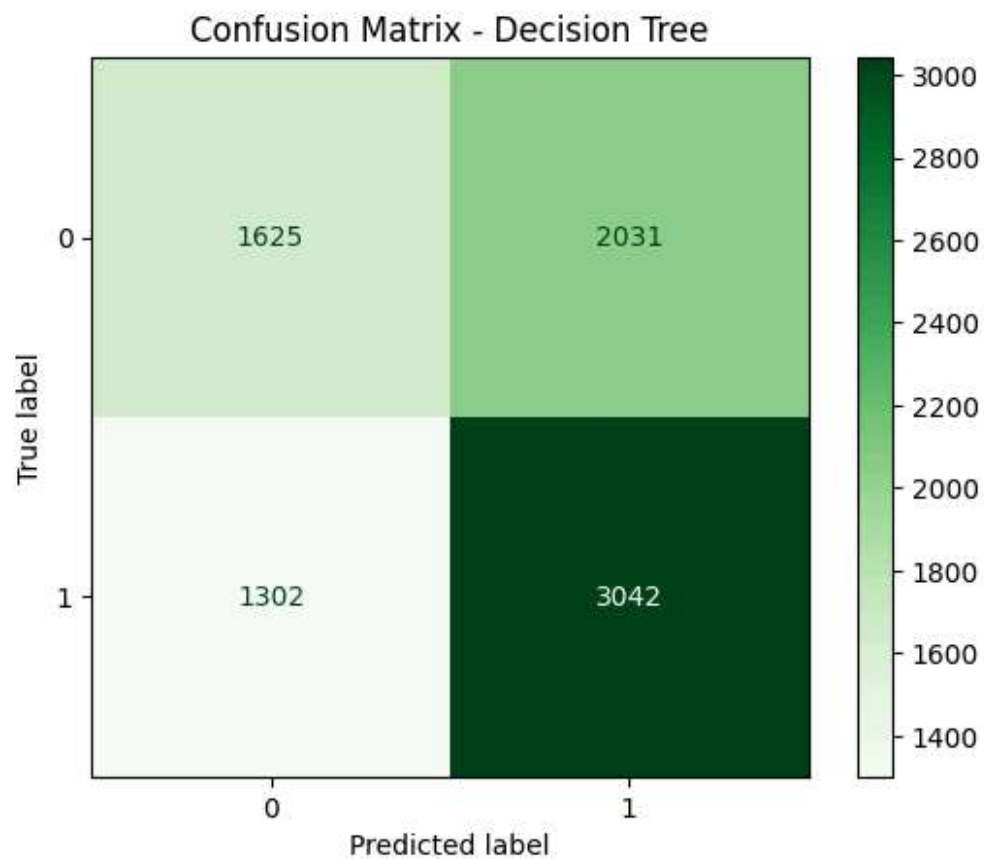
```

```

Accuracy of Decision tree : 0.583375
Classification Report of Decision Tree :

```

	precision	recall	f1-score	support
0	0.56	0.44	0.49	3656
1	0.60	0.70	0.65	4344
accuracy			0.58	8000
macro avg	0.58	0.57	0.57	8000
weighted avg	0.58	0.58	0.58	8000



XGBoost

```
In [ ]: from xgboost import XGBClassifier
        from sklearn.metrics import accuracy_score, classification_report,
        confusion_matrix
```

Importing the required libraries for the XGBoost model.

```
In [ ]: xgb_model = XGBClassifier(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=5,
        random_state=42,
        eval_metric='logloss'
    )

xgb_model.fit(X1_train, Y_train)
```

▼ XGBClassifier ⓘ ?

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None,
              early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None,
              gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=0.1,
              max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
```

These are required parameters for XGBoost

`n_estimators = 100` (builds 100 decision trees one after another)

`learning_rate = 0.1` (each tree contributes 10% to the final result)

`max_depth = 5` (each tree can go 5 levels deep)

`eval_metric = logloss` (uses log loss to measure error during training)

```
In [ ]: Y_pred_xgb = xgb_model.predict(X1_test)
```

The learned model has been predicting using test data.

Result of XGBoost

```
In [ ]: print("Accuracy of XGBoost", accuracy_score(Y_test, Y_pred_xgb))

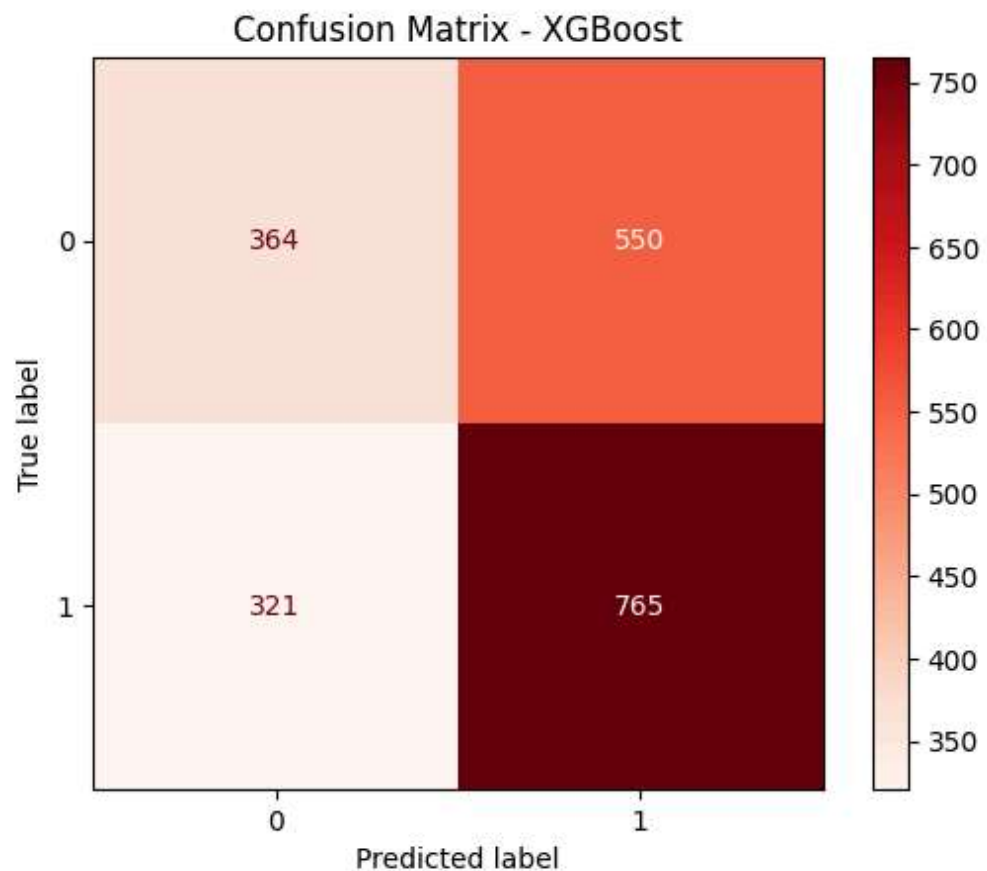
print("\nClassification Report of XGBoost:\n",
      classification_report(Y_test, Y_pred_xgb))

cm = confusion_matrix(Y_test, Y_pred_xgb)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Reds')
plt.title("Confusion Matrix - XGBoost")
plt.show()
```

Accuracy of XGBoost 0.5645

Classification Report of XGBoost:

	precision	recall	f1-score	support
0	0.53	0.40	0.46	914
1	0.58	0.70	0.64	1086
accuracy			0.56	2000
macro avg	0.56	0.55	0.55	2000
weighted avg	0.56	0.56	0.55	2000



Regression

Data preprocessing for Regression

```
In [ ]: from sklearn.model_selection import train_test_split
        from sklearn.model_selection import RandomizedSearchCV
```

Uploading necessary libraries for our next random forest model.

```
In [ ]: PS1 = PS[PS['placement_status'] == 'Placed']

X_reg = PS1.drop(['student_id', 'placement_status', 'salary_package_lpa'],
axis=1)
Y_reg = PS1['salary_package_lpa']

print(X_reg.shape)
print(Y_reg.shape)
```

```
(5430, 23)
(5430,)
```

Here only we work with placed students so we create new data only with placed students,

Because here we are going to work with "salary_package_lpa" and this is only for placed students hence we create "PS1" new data set


```
In [ ]: X_reg = PS1.drop(['student_id', 'placement_status', 'salary_package_lpa'],  
axis=1)  
X_reg
```

	age	gender	cgpa	branch	college_tier	internships_count
2	22	Female	8.60	EEE	Tier 1	0
4	20	Female	8.43	IT	Tier 3	1
5	22	Male	7.44	CSE	Tier 2	1
8	19	Male	8.52	Mechanical	Tier 2	2
9	20	Female	8.07	Civil	Tier 2	2
...	...	...	...	...	...	...
9987	20	Female	8.35	ECE	Tier 2	1
9989	24	Female	8.32	IT	Tier 2	2
9993	20	Male	7.04	Civil	Tier 2	2
9996	22	Male	7.59	CSE	Tier 2	2
9999	19	Female	7.88	CSE	Tier 1	0

5430 rows × 23 columns



We remove outputs columns and create new variable for only inputs.

```
In [ ]: Y_reg = PS1[['salary_package_lpa']]
        Y_reg
```

	salary_package_lpa
2	11.99
4	12.16
5	11.78
8	16.33
9	16.17
...	...
9987	12.63
9989	15.35
9993	11.17
9996	13.18
9999	14.55

5430 rows × 1 columns

This is for output column.

```
In [ ]: X_reg = pd.get_dummies(X_reg, columns=['gender', 'branch', 'college_tier',
        'volunteer_experience'], drop_first=True)

        X_reg = X_reg.astype(float)

        print(X_reg.dtypes.unique())
        print(X_reg.isnull().sum().sum())
```

```
[dtype('float64')]
0
```

To give only float values to all columns in this we use get_dummies.

```
In [ ]: X_train1, X_test1, Y_train1, Y_test1 = train_test_split(X_reg, Y_reg,
        test_size=0.2, random_state=42)
```

Splitting up the data with test and train datas for model.

Random Forest Regressor

```
In [ ]: from sklearn.ensemble import RandomForestRegressor
        from sklearn.metrics import r2_score, mean_squared_error,
        mean_absolute_error
```

Importing required libraries.

```
In [ ]: RF_reg = RandomForestRegressor(n_estimators=100,
                                     max_depth=10,
                                     min_samples_split=5,
                                     min_samples_leaf=2,
                                     random_state=42,
                                     n_jobs=-1)

RF_reg.fit(X_train1, Y_train1)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/base.py:1389:
DataConversionWarning: A column-vector y was passed when a 1d
array was expected. Please change the shape of y to (n_samples,),
for example using ravel().
    return fit_method(estimator, *args, **kwargs)
```

▼

RandomForestRegressor

RandomForestRegressor(max_depth=10, min_samples_leaf=2,
min_samples_split=5,
n_jobs=-1, random_state=42)

Initilize the RandomForestRegressor with these parameters and then Fit into the model

`n_estimators = 100` (Build 100 decision trees)

`max_depth = 10` (Each tree can go maximum 10 levels deep)

`min_samples_split = 5` (Need minimum 5 samples to split a node)

`min_samples_leaf = 2` (Each leaf must have minimum 2 samples)

`n_jobs = -1` (Use all CPU cores — runs faster)

```
In [ ]: Y_pred_test_reg = RF_reg.predict(X_test1)
```

Predicting the test model using the learned train data set.

```
In [ ]: print("MAE:", mean_absolute_error(Y_test1, Y_pred_test_reg))
        print("MSE:", mean_squared_error(Y_test1, Y_pred_test_reg))

        print("R2 Score :", r2_score(Y_test1, Y_pred_test_reg))
        print("RMSE      :", np.sqrt(mean_squared_error(Y_test1, Y_pred_test_reg)))
```

```
MAE: 0.8531202986933104
MSE: 1.3317250303727615
R2 Score : 0.5061073917920784
RMSE      : 1.1540039126332118
```

XGBoost Regressor

```
In [ ]: from xgboost import XGBRegressor
        from sklearn.metrics import mean_absolute_error, mean_squared_error,
        r2_score
        import numpy as np
```

Importing required libraries.

```
In [ ]: xgb_reg = XGBRegressor(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=5,
        random_state=42
    )

xgb_reg.fit(X_train1, Y_train1)
```

▼ XGBRegressor ⓘ ?

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None,
              early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None,
              feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.1, max_bin=None,
              max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None,
```

These are required parameters for XGBoost Regressor

`n_estimators = 100` (builds 100 decision trees one after another)

`learning_rate = 0.1` (each tree contributes 10% to the final result)

`max_depth = 5` (each tree can go 5 levels deep)

```
In [ ]: Y_pred_xgb_reg = xgb_reg.predict(X_test1)
```

The model learned already using train data now predicting using test data.

Result of XGBoost Regressor

```
In [ ]: print("MAE:", mean_absolute_error(Y_test1, Y_pred_xgb_reg))
        print("MSE:", mean_squared_error(Y_test1, Y_pred_xgb_reg))
        print("RMSE:", np.sqrt(mean_squared_error(Y_test1, Y_pred_xgb_reg)))
        print("R2 Score:", r2_score(Y_test1, Y_pred_xgb_reg))
```

```
MAE: 0.8545691967010498
MSE: 1.3327847719192505
RMSE: 1.1544629798825299
R2 Score: 0.5057144165039062
```

Gradient Boosting Regressor

```
In [ ]: from sklearn.ensemble import GradientBoostingRegressor
```

Importing libraries.

```
In [ ]: gb_reg = GradientBoostingRegressor(
          n_estimators=100,
          learning_rate=0.1,
          max_depth=5,
          random_state=42
        )

gb_reg.fit(X_train1, Y_train1)
```

```
/usr/local/lib/python3.12/dist-
packages/sklearn/ensemble/_gb.py:672: DataConversionWarning: A
column-vector y was passed when a 1d array was expected. Please
change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True) # TODO: Is this still required?
```

▼

GradientBoostingRegressor




GradientBoostingRegressor(max_depth=5, random_state=42)

These are required parameters for Gradient XGBoost Regressor

`n_estimators` = 100 (builds 100 decision trees one after another)

`learning_rate` = 0.1 (each tree contributes 10% to the final result)

`max_depth` = 5 (each tree can go 5 levels deep)

```
In [ ]: Y_pred_gb_reg = gb_reg.predict(X_test1)
```

The model already learnt by using that it is predicting the test data.

Result of Gradient XGBoost Regressor

```
In [ ]: print("MAE:", mean_absolute_error(Y_test1, Y_pred_gb_reg))
        print("MSE:", mean_squared_error(Y_test1, Y_pred_gb_reg))
        print("RMSE:", np.sqrt(mean_squared_error(Y_test1, Y_pred_gb_reg)))
        print("R2 Score:", r2_score(Y_test1, Y_pred_gb_reg))
```

```
MAE: 0.8478756267507755
MSE: 1.3124948067106554
RMSE: 1.1456416571994297
R2 Score: 0.5132392434163141
```

Conclusion

We have build 3 model for the classification and regression and lets find which model is optimal and effeciency for this project

Classification summary

- * Accuracy of Logistic Regression: 0.501625
- * Accuracy of Decision Tree = 0.583375
- * Accuracy of XGBoost 0.5645

The Decision Tree model give us optimal accuracy comparatively, and we can build further model using Decision tree for this project

Regression summary

* Random Forest:

MAE : 0.8531202986933104

MSE : 1.331725030372762

RMSE : 1.154003912633212

R2 Score : 0.5061073917920784

* XGBoost:

MAE : 0.8545691967010498

MSE : 1.3327847719192505

RMSE : 1.1544629798825299

R2 Score : 0.5057144165039062

* Gradient XGBoost:

MAE : 0.8478756267507755

MSE : 1.3124948067106554

RMSE : 1.1456416571994297

R2 Score : 0.5132392434163141

We can choose the optimal model based on the three factors they are

* R2 score = That must be closer to 1

* RMSE = That must be as lower as lower

* MAE = This must be as lower as lower

Based upon these factors we analysis that Gradient XGBoost has the best model with

* R2 score = 0.5132

* RMSE = 1.1456

* MAE = 0.8478

Therefor for the regression we can use Gradient XGBoost Regressor of the model building for this project

The final conclusion that states

""Based upon the given data we had analysis the performance of the students and their salary package , By using this model the university can easily predict the student placement percentage according to thier performance in academics and exter curricular activities"

To achive this we had used two machine learning algorithm for the model building and for classification we use **Decision Tree** and for Regression we use **Gradient XGBoost Regressor**

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).